

xv6-riscv LoongArch HAL

项目设计文档

xiao-yang-lab

2026 年 6 月 13 日

目录

1 项目概述

本项目为 MIT xv6-riscv 第 5 版教学操作系统引入硬件抽象层 (HAL)，使其能够在 RISC-V 和 LoongArch (LA64) 两个处理器架构上运行。RISC-V 侧的 HAL 已全部完成 (通过完整 usertests)，LoongArch 侧经过多轮调试实现了单核 HAL 并通过 usertests -q 全量测例。

1.1 核心目标

- 将平台相关代码与内核通用逻辑解耦
- 建立统一的跨平台 HAL 接口
- RISC-V 和 LoongArch 均可通过 test-xv6.py -q usertests

1.2 项目结构

```
xv6-riscv/
  kernel/          # 通用内核代码 (尽量不动)
  hal/
    hal.h          # HAL 统一接口
    hal_arch.h     # CPU 控制接口
    hal_vm.h       # 虚拟内存接口
    hal_intr.h     # 中断控制器接口
    hal_timer.h    # 定时器接口
    hal_console.h  # 控制台 I/O 接口
    hal_ctx.h      # 上下文切换接口
    riscv/         # RISC-V 平台实现
    loongarch/     # LoongArch 平台实现
  Makefile         # ARCH=riscv|loongarch 选择
```

1.3 平台选择与编译

通过 ARCH 编译变量选择平台：

- ARCH=riscv 使用 hal/riscv/ 下的实现，QEMU 用 qemu-system-riscv64
- ARCH=loongarch 使用 hal/loongarch/ 下的实现，QEMU 用 qemu-system-loongarch64

LoongArch 内核启动使用 -bios kernel/kernel.bin。make qemu 会先通过 objcopy -O binary 将 ELF kernel 转成 raw binary。LoongArch 工具链为 loongarch64-linux-gnu-，编译参数使用 -march=loongarch64 -mcmmodel=normal。

2 LoongArch HAL 整体设计思路

2.1 架构差异与 HAL 抽象策略

LoongArch 与 RISC-V 存在 6 个架构级关键差异，需要通过 HAL 统一接口屏蔽。

表 1: 架构差异与 HAL 抽象

维度	RISC-V	LoongArch	HAL 方案
特权级	M/S/U 三级	PLV0/PLV3 两级	SSTATUS $\leftrightarrow$ PRMD 转换
页表 walk	硬件 Sv39	软件 TLB 重填	hal_tlb_flush_all() + TLBREENTRY
TLB 刷新	sfence.vma	invtlb	统一 sfence_vma() 宏
异常返回	sret/mret	ertn	trampoline.S 中用 ertn
中断控制器	PLIC	EIOINTC	hal_irq_* 接口
定时器	CLINT MMIO	TCFG/TVAL CSR	hal_timer_* 接口
virtio 磁盘	MMIO @0x10001000	PCI virtio	RAM-disk 替代方案
Callee-saved	14 寄存器	12 寄存器	hal_ctx.h 条件编译

2.2 设计原则

1. 最小化通用代码修改：内核通用逻辑（kernel/）只调用 hal_ 前缀接口，不直接依赖平台 CSR。
2. 复用 RISC-V 接口：LoongArch 的 arch.h 提供 RISC-V 兼容层（w_satp→w_pgdl, w_stvec→w_eentry 等），trap.c 无需修改。
3. ESTAT→scause 转换：在 estat_to_scause() 中将 LoongArch 异常码映射为 RISC-V 的 scause 值，避免直接透传导致冲突（如 LA FPD=0xf 与 RV store fault 冲突）。
4. DMW 直通内核空间：内核通过 DMW0（VSEG=0, PLV0=1, MAT=CC）恒等映射访问所有物理地址，无需 TLB。
5. 跨架构兼容性：页表层数通过 PT_LEVELS 宏区分（RISC-V=3, LoongArch=4）；PTE 有效位通过 PTE_V_CACHE 统一。

2.3 RAM-disk 块设备策略

LoongArch QEMU virt 的 virtio 走 PCI 设备模型，与 RISC-V 的 MMIO virtio 不同。为优先跑通 HAL 和文件系统，LoongArch 当前不实现完整 PCI 枚举，而是采用内嵌 RAM disk 方案：

1. 构建时通过 `objcopy -I binary` 将 `fs.img` 转为 ELF object
2. 链接时将镜像并入内核
3. 启动时 HAL 将只读镜像内容复制到可写内存页
4. 上层 `bio.c` 仍通过 `bread(dev, blockno) / bwrite()` 访问块设备

同时支持双 RAM-disk (`VIRTIO_NDISK=2`)，`dev=1` 对应 `fs.img`，`dev=2` 对应 `fs1.img`。

3 关键实现描述

3.1 内存布局 (`memlayout.h`)

LoongArch QEMU virt 机器的物理地址布局与 RISC-V 不同。内核代码加载在 flash (`0x1C000000`)，可写数据段 (`.data/.bss`) 放在 RAM (`0x00400000`)。start() 负责在启动时将 `.data` 从 flash 搬运到 RAM，并清零 `.bss`。

Listing 1: `memlayout.h` 关键定义

```

1 #define UART0      0x1FE001E0L      // 16550a 串口基址
2 #define EIOINTC    0x0FE00000L      // 扩展 I/O 中断控制器
3 #define KERNBASE   0x1C000000L      // 内核 flash 基址
4 #define PHYSTOP    (0x00400000L + 124*1024*1024) // 可管理 RAM
   上限
5 #define TRAMPOLINE 0x1C009000L      // 陷入/返回代码页 (flash
   中)
6 #define TRAPFRAME  (MAXVA - 2*PGSIZE) // 用户态 trapframe VA (保
   持 xv6 契约)
7 #define USER_TOP   TRAPFRAME        // 用户堆增长上限

```

3.2 CPU 控制与 PTE 格式 (`arch.h`)

`arch.h` 是 LoongArch HAL 的核心头文件，约 460 行，包含：

- CSR 寄存器地址宏和读写内联函数 (`csrrd/csrwr`)
- CRMD/PRMD/ECFG/ESTAT 位域定义
- 中断控制内联函数 (`intr_on/off/get`)
- PTE 格式宏 (`V/D/PLV/MAT/P/W/PPN` 位序完全按手册 Figure 8)
- `estat_to_scause()` 异常码转换

- RVACFG 支持（降低硬件有效 VA 位宽）
- RISC-V 兼容层别名

Listing 2: PTE 格式宏定义

```

1 #define PTE_V      (1L << 0)      // valid
2 #define PTE_D      (1L << 1)      // dirty / writable
3 #define PTE_PLV0    (0L << 2)      // kernel-only
4 #define PTE_PLV3    (3L << 2)      // user-mode
5 #define PTE_MAT      (1L << 4)      // MAT=01: cacheable coherent
6 #define PTE_G      (1L << 6)      // global
7 #define PTE_HW_P    (1L << 7)      // hardware walker present flag
8 #define PTE_HW_W    (1L << 8)      // hardware walker writable flag
9
10 // Leaf PTE: V + MAT(CC) + P + W
11 #define PTE_V_CACHE (PTE_V | PTE_MAT | PTE_HW_P | PTE_HW_W)
12
13 // PPN extraction (manual §5.4.5, Figure 8)
14 #define PAMASK      0xFFFFFFFFF000UL
15 #define PA2PTE(pa)  (((uint64)(pa)) & PAMASK)
16 #define PTE2PA(pte) (((uint64)(pte)) & PAMASK)
17
18 // 4-level page table indices
19 #define PXSHIFT(level) (PGSHIFT+(9*(level)))
20 #define PX(level, va) (((uint64)(va)) >> PXSHIFT(level)) &
    PXMASK)
21
22 #define MAXVA (1ULL << 38)      // same as xv6 Sv39
23 #define PT_LEVELS 4              // LoongArch uses 4-level page
    tables

```

3.3 页表 walk (vm.c)

通用 walk() 函数通过 PT_LEVELS 宏适配不同架构的页表层数 (RISC-V Sv39=3 级, LoongArch=4 级), lazy allocation 使用 PTE_V_CACHE 创建 PTE。

Listing 3: walk() 层数适配

```

1 for(int level = PT_LEVELS - 1; level > 0; level--) {
2     pte_t *pte = &pagetable[PX(level, va)];
3     if(*pte & PTE_V) {
4         pagetable = (pagetable_t)PTE2PA(*pte);

```

```

5     } else {
6         if(!alloc || (pagetable = kalloc()) == 0)
7             return 0;
8         memset(pagetable, 0, PGSIZE);
9         *pte = PA2PTE(pagetable) | PTE_V;
10    }
11 }
12 return &pagetable[PX(0, va)];

```

3.4 TLB 重填处理器 (`hal_tlbrefill.S`)

这是整个 LoongArch 移植中最关键的组件。

LoongArch MMU 在 TLB miss 时不自动 walk 页表，而是触发 TLB 重填异常 (TLBR)，通过 TLBREENTRY (CSR 0x88) 跳转到软件 handler。Handler 在 DA=1 (直接地址翻译) 模式下运行，使用 lddir/ldpte 指令 walk 4 级页表。

Listing 4: TLB refill handler 核心逻辑

```

1 tlb_refill_entry:
2     csrwr    $t0, 0x8b                # save t0 to TLBRSAVE
3
4     # set TLBREHI.PS = 12 (4KB pages)
5     csrrd    $t0, 0x8e
6     srli.d   $t0, $t0, 6
7     slli.d   $t0, $t0, 6
8     ori      $t0, $t0, 12
9     csrwr    $t0, 0x8e
10
11    # MAXVA boundary check: reject VA >= 1<<38 and VA < 2*PGSIZE
12    csrrd    $t0, 0x89                # TLBRBADV
13    srli.d   $t0, $t0, 38
14    bnez     $t0, .Ltlbr_invalid
15    csrrd    $t0, 0x89
16    srli.d   $t0, $t0, 13
17    beqz     $t0, .Ltlbr_invalid
18
19    # 4-level lddir walk: L3(VA[47:39]) → L2(VA[38:30]) → L1(VA
20    [29:21])
21    csrrd    $t0, 0x19                # PGDL
22    lddir    $t0, $t0, 3              # L3
23    andi     $t0, $t0, 1              # check V

```

```

23     beqz      $t0, .Ltlbr_invalid
24     csrrd     $t0, 0x19                # re-read PGDL, walk L3 again
25     lddir     $t0, $t0, 3
26     addi.d    $t0, $t0, -1            # clear V flag
27     # ... L2, L1 similarly
28
29     # leaf PTE → TLBRELO0/1
30     ldpte     $t0, 0
31     ldpte     $t0, 1
32
33     # clear memory PTE P(b7)/W(b8) — reserved in TLBELO
34     csrrd     $t0, 0x8c
35     bstrins.d $t0, $r0, 8, 7
36     csrwr     $t0, 0x8c
37     csrrd     $t0, 0x8d
38     bstrins.d $t0, $r0, 8, 7
39     csrwr     $t0, 0x8d
40
41     tlbfill                    # fill random TLB slot
42     csrrd     $t0, 0x8b        # restore t0
43     ertn

```

设计要点：

1. 逐级检查 V 位，任何一级无效就走 invalid 路径
2. 每级 walk 前从 PGDL 重新走整条路径（因只剩一个临时寄存器，无法保存中间 base）
3. ldpte 后清除 P/W 位（手册 §5.4.5 明确内存 PTE 的 bit7/8 在 TLBELO 中是保留位）
4. 在 lddir 之前做 MAXVA 边界拒绝（防止高地址被截断后别名到低地址）

3.5 用户态陷入/返回 (hal_trampoline.S)

约 140 行汇编，实现 uservec/userret。

关键设计：LoongArch 没有 sscratch，改用 KSave0 (CSR 0x30) 和 KSave1 (CSR 0x31)：

- KSave0 保存/恢复用户 a0
- KSave1 保存当前进程 trapframe 的内核虚拟地址 (DMW0 可访问)

陷入时 uservec 通过 KSave1 获取 trapframe，将全部用户寄存器保存到 trapframe，切换页表到内核 PGDL 后跳转 usertrap。返回时 userret 恢复寄存器，切回用户

PGDL 后 `ertn`。

3.6 上下文切换 (`hal_swch.S`)

LoongArch callee-saved 为 12 个寄存器 (`ra, sp, fp, s0-s8`)，比 RISC-V (14 个: `ra, sp, s0-s11`) 少 2 个。`hal_ctx.h` 通过 `#ifdef ARCH_loongarch` 条件编译不同结构体。

3.7 中断控制器 (`hal_intr.c`)

LoongArch 使用 EIOINTC (MMIO 基址 `0x0FE00000`)，与 RISC-V PLIC 完全不同。64-bit ISR 位图管理中断，`hal_irq_claim()` 采用简单位扫描获取最高优先级 IRQ。`hal_irq_complete()` 只需在 `claim` 时清除 ISR 位。

3.8 MMU 初始化 (`kvmnithart` in `vm.c`)

在 `kvmnithart()` 中完成 LoongArch MMU 的完整初始化：

Listing 5: `kvmnithart` LoongArch 关键配置

```

1 // DMW0: PLV0 only, VSEG=0, MAT=CC → VA[63:60]=0 identity-map to
   PA
2 w_dmw0(0x000000000000000011ULL);
3 w_dmw1(0);
4 sfence_vma();
5
6 // 4-level page table: 9-9-9-9-12 (must match walk() and TLB
   handler!)
7 asm volatile("csrwr %0, 0x1C" : : "r"( // PWCL
8     (12UL<<0) | (9UL<<5) | (21UL<<10) | (9UL<<15) | (30UL<<20) | (9UL<<25)
9 ));
10 asm volatile("csrwr %0, 0x1D" : : "r"( // PWCH
11     (39UL<<0) | (9UL<<6)
12 ));
13
14 // Reduce hardware effective VA for xv6 compatibility (MAXVA
   =1<<38)
15 w_rvacfg(8);
16
17 // TLB refill handler entry
18 extern void tlb_refill_entry(void);

```



```

19 uint64 tlbr_entry = (uint64)tlb_refill_entry;
20 asm volatile("csrwr %0, 0x88" : "+r"(tlbr_entry));
21
22 // Enable MMU: DA=0, PG=1
23 uint64 crmd = r_crmd();
24 crmd &= ~CRMD_DA;
25 crmd |= CRMD_PG;
26 w_crmd(crmd);

```

3.9 启动与数据重定位 (hal_start.c)

QEMU -bios 将内核二进制加载到只读 flash (0x1C000000)。内核的 .data 和 .bss 必须放在可写 RAM (0x00400000)。start() 负责：

1. 仅 0 号核复制 .data、清零 .bss，其他核等待 boot_done
2. 设置 tp=cpuuid (多核区分)
3. 设置 EENTRY 指向 kernelvec
4. 使能 FPU (CSR EUEN)
5. 进入 main()

3.10 其他关键修改

- kernel/exec.c: LoongArch 下预映射低 2 页为非用户 guard (PTE_U=0)
- kernel/trap.c: vmfault 增加 scause=12 (指令缺页) 处理; estat_to_scause 转换中 ADE/IPE 等避免与 RISC-V 值冲突
- kernel/proc.c: growproc 使用 USER_TOP 统一边界
- hal/loongarch/user.ld: 用户程序从 0x2000 链接
- hal/loongarch/kernel.ld: tlbrefill 固定在 0x1c008000, trampoline 在 0x1c009000

4 研发过程中的问题与解决方法

4.1 环境搭建阶段 (第 4 周)

1. Docker 工具链版本: Ubuntu 24.04 apt 中无元包, 实际包名 gcc-14-loongarch64-linux-gnu, 需手动建立符号链接。
2. QEMU -bios 只接受 raw binary: 需 objcopy -O binary
3. QEMU LoongArch virt 至少 1GB RAM (RISC-V 只需 128MB)
4. -mmodel 差异: LoongArch 裸机程序用 normal (RISC-V 用 medany)

5. Makefile 变量覆盖顺序：LoongArch 的 TOOLPREFIX/QEMU 设置需放在 ifn-def 之前

4.2 基础 HAL 实现阶段（第 5 周）

6. PA2PTE 类型错误：宏参数可能是指针类型 (uint64*)，C 不允许指针对位运算。在宏中添加 (uint64) 显式转换。
7. RISC-V CSR 兼容：内核通用代码中仍有多处直接调用 RISC-V CSR 函数 (w_satp, w_stvec, r_sstatus 等)。在 arch.h 中添加 #define w_satp w_pgdl 等兼容别名。
8. SSTATUS→PRMD 转换：RISC-V sstatus.SPP/SPIE 与 LoongArch PRMD.PPLV/PIE 位序完全不同。实现 r_sstatus_compat() 和 w_sstatus_compat() 转换函数。
9. PTE PPN 分两段存储：LoongArch PPN[47:0]→PTE[59:12], PPN[52:48]→PTE[11:7]，与 RISC-V 连续存储不同。

4.3 中断/上下文切换阶段（第 6 周）

10. .data/.bss 在只读 flash 中（最关键的早期阻塞）：QEMU -bios 将整个内核二进制加载到只读 flash。内核写 .bss 中全局变量（如 spinlock）时触发存储异常 → kerneltrap panic。解决：链接脚本分离 flash (.text/.rodata) 和 RAM (.data/.bss)，start() 将 .data 从 flash 搬运到 RAM，清零 .bss。
11. 定时器中断风暴：kernelvec 最初只有 ertn，未清除定时器中断标志 (TICLR)，CPU 反复响应同一中断陷入无限循环。解决：clockintr() 中调用 w_ticlr(0)。
12. UART QEMU 兼容性：波特率重编程破坏输出、FCR 重置触发 QEMU 断言崩溃、TX idle 轮询不可靠。解决：uartinit() 跳过波特率/FCR 操作。
13. kernelvec 寄存器保存不全：最初只保存 ra/tp，tp 保存后未恢复。解决：完整保存/恢复 17 个 caller-saved 寄存器。
14. virtio 磁盘：LoongArch QEMU virt 的磁盘是 PCI virtio 而非 MMIO。解决：RAM-disk 替代方案——构建时将 fs.img 通过 objcopy 嵌入内核，启动时复制到可写内存页。

4.4 整合调试阶段（第 7 周）

15. TLB refill handler 3 级/4 级不匹配（最核心的技术挑战）：walk() 创建 4 级页表，但 TLB handler 最初是 3 级 walk。解决：统一为 4 级，使用 lddir/ldpte 标准指令 + PWCL/PWCH 显式配置。
16. TLB entry P/W 位污染：内存 PTE 的 bit7(P)/bit8(W) 在 TLBELO 中是保留位，直接写入导致 TLB 条目损坏。解决：ldpte 后用 bstrins.d 清零。

17. TLBRBADV vs BADV: TLB refill handler 应读 TLBRBADV (0x89) 而非 BADV (0x7)。
18. MAXVAplus 测试失败 (最终阻塞): MAXVA 从 $1 \ll 38$ 放宽到 $1 \ll 46$ 导致 xv6 地址契约被破坏, 高地址在 LoongArch 48-bit VA 下被截断后别名到低地址。解决: 恢复 MAXVA= $1 \ll 38$ + TLB 入口边界拒绝 + exec 低 2 页 guard + RVACFG=8 三层防护。
19. RISC-V 编译 LoongArch TLB 汇编失败: 通用 vm.c 中包含 LoongArch 专用 CSR/TLB 汇编, 缺少条件编译。解决: 加 `#ifdef ARCH_loongarch`。
20. PTE_V_CACHE 跨架构未定义: LoongArch 定义了该宏但 RISC-V 未定义。解决: RISC-V 侧退化为 PTE_V。
21. 页表层数写死: walk() 曾按 LoongArch 4 级写死。解决: 引入 PT_LEVELS (RISC-V=3, LoongArch=4)。

4.5 关键技术难点分析

表 2: 之前的误区 vs 正确的做法

维度	之前的误区	正确的做法
TLB walk 方法	自己写 ld.d 软 walk, 3 级/4 级切换出错	用 lddir/ldpte 标准指令 + PWCL/PWCH 显式配置
MAXVA 策略	放宽 MAXVA 以适配 LoongArch 宽 VA	严格保持 xv6 原始 MAXVA= $1 \ll 38$, 多层防护
TLB entry 格式	内存 PTE 直接写 TLBELO	清除 P/W 保留位后写入
调试方法	vmfault 手工填 TLB $\rightarrow$ userret 冲掉	直接 fix TLB handler 本身
问题定位	在症状层面加 patch (dbar/invtlb/PWCL)	追根因: 页表结构与 handler 不一致
开发流程	从 broken commit 出发反复 patch	系统梳理 LoongArch 手册, 用 lddir 标准方案重写

5 与参考项目 xv6-loongarch-exp-main 的差异

项目早期参考了竞赛项目 xv6-loongarch-exp-main。但后续发现该参考项目使用的 QEMU 环境 (RAM 在 0x90000000, 使用 `-kernel` UEFI 启动) 与竞赛要求的标准 QEMU virt 机器 (RAM 在 0x00000000, `-bios` 裸机启动) 完全不同。因此不能直接套用其代码。

重要说明：PTE 格式 (V/D/PLV/MAT/P/W 位序)、4 级页表结构、lddir/ldpte 指令用法、PWCL/PWCH 配置等是 LoongArch 架构手册规定的标准做法，不是参考项目独有的设计。两个项目在这些方面的代码相似是因为都遵循了同一份架构规范。

5.1 两个项目的本质差异

表 3: 参考项目 vs 本项目：核心差异一览

差异项	参考项目 (exp-main)	本项目 (最终代码)	
QEMU 物理 RAM 位置	0x90000000+	0x00000000- 0x0FFFFFFFF (标准 virt 机器)	
DMW 窗口	VSEG=0x9, DMW0=0x9000000000000000	VSEG=0, DMW0=0x0000000000000000011	
物理地址访问	pa DMWIN_MASK (0x9000000000000000)	直接使用原始 PA, DMW0 恒等映射	
内核链接地址	0x9000000000200000	0x1C000000 (flash 复位向量)	
启动方式	-kernel (UEFI 加载 ELF)	-bios (裸机 raw binary)	
内存初始化	无需搬运.data/.bss (UEFI 已做)	start() 手动将.data 从 flash 复制到 RAM、清零.bss	
内核栈位置	TRAPFRAME 下方 (高 VA)	0x08000000+ (低物理地址, DMW0 可达)	
trapframe 访问	CSR.SAVE 保存用户 VA 映射	KSave1 保存内核 VA (DMW0 可达)	
TRAMPOLINE VA	高 VA (用户页表映射)	低 VA = 0x1C009000L (PLV0 DMW0 直接访问)	
块设备	RAM-disk (单盘)	RAM-disk (支持双盘, VIRTIO_NDISK=2)	
页表层数抽象	写死 4 级	PT_LEVELS 宏 (RISC-V=3, LoongArch=4), 跨架构兼容	

5.2 本项目独有的设计

以下功能是本项目在参考项目基础上自主增加或改进的，参考项目代码中不存在这些实现：

表 4: 本项目独有的设计

独有功能	说明
TLB handler MAXVA 边界拒绝	在 lddir 前检查 $TLBRBADV \geq 1 \ll 38$ 和 $TLBRBADV < 2 * PGSIZE$ ，拒绝非法 VA 进入页表 walk。参考项目无此检查
TLBELO P/W 位清除	ldpte 后用 bstrins.d 清零 bit7(P) 和 bit8(W)。手册规定这些位在 TLBELO 中是保留位，不清除会污染 TLB 条目。参考项目未做此处理
RVACFG 硬件 VA 限制	写 RVACFG=8 降低硬件有效 VA 位宽，配合 MAXVA 软件检查形成双层防护。参考项目无此配置
exec 低页 guard	LoongArch 下预映射低 2 页为 PTE_U=0（非用户页），防止高地址被硬件截断后别名到低地址命中用户数据。用户程序从 0x2000 链接运行
跨架构 PT_LEVELS / PTE_V_CACHE	RISC-V 和 LoongArch 共用 vm.c，通过 PT_LEVELS（3 vs 4）和 PTE_V_CACHE 宏区分。参考项目写死 4 级页表
双 RAM-disk 支持	VIRTIO_NDISK=2 时链接 fs1_img.o，dev=2 访问第二块设备。参考项目仅支持单盘
estat_to_scause 异常码转换完善	ADE (ecode=8) 根据 EsubCode 分派为 scause=12 或 13；IPE (ecode=14) 映射为 scause=12 而非 2；default 返回 0x100 ecode 而非透传。避免 LoongArch ecode 与 RISC-V scause 的冲突
flash/RAM 分离的 链接布局	start() 在裸机环境下手动搬运.data 初始值、清零.bss。参考项目依赖 UEFI 完成此工作

6 AI 工具使用声明

本项目允许使用 AI 工具辅助开发。

6.1 使用的 AI 工具

- 1. Claude Code + DeepSeek-V4 API：通过 CLI 交互方式使用，负责大部分代码书写和调试辅助。在开发过程中完成了 RISC-V HAL 的代码生成和 LoongArch

HAL 初期实现，但在 TLB refill handler 问题上长期未能提出有效方案。

- 2. OpenAI Codex CLI：在开发者自行查阅 LoongArch 技术手册、梳理 TLB refill 和 MAXVA 等关键问题的解决思路后，由 Codex 完成 LoongArch HAL 的最终实现和调试通过。

6.2 AI 工具使用场景与成果总览

表 5: AI 工具使用场景与成果

使用场景	AI 辅助内容	最终成果
架构差异分析	对比 RISC-V 与 LoongArch 手册，整理 CSR/PTE/异常码对照表	开发者验证并修正后纳入 CLAUDE.md
TLB refill handler 设计	提出软件 walk 方案，讨论 3 级/4 级匹配问题	多次尝试未解决问题，最终由开发者查阅手册后改用 ld-dir/ldpte 方案
PTE 格式验证	逐位对照 LoongArch 手册 Figure 8 验证宏定义	确认与手册一致
内存布局设计	分析 QEMU virt 物理地址布局，建议 flash/RAM 分离方案	开发者采纳并实现
DMW 配置分析	对比参考项目与标准 QEMU 配置，确认 VSEG=0 正确性	开发者确认
调试辅助	fork-chk 内核读写验证、copyinstr 物理地址追踪	帮助定位问题方向
跨架构兼容	分析 PT_LEVELS/PTE_V_CACHE 跨架构问题	开发者审核后合入
块设备设计	RAM-disk 替代方案、双磁盘支持	AI 建议方案，开发者实现
文档撰写	CLAUDE.md、HANDOFF.md、分析文档	开发者审核修改后采纳

6.3 AI 交互记录节选

以下为本次项目中与 AI 工具的关键交互记录。每条记录包含：开发者提问、AI 回答、开发者采纳实施结果、以及（若失败）开发者后续的正确思路。

交互 1：非叶 PTE 缺少 P 位（错误方向）

- 1) 开发者提问：copyinstr2 测试失败，子进程 BSS 写入在内核 copyinstr 中读到 0。怀疑 TLB handler 填入了错误的物理页。查阅手册发现 Figure 8 中 P 字段 (bit7) “在页表 walk 过程中使用”，是否需要在非叶 PTE 中设置 PTE_HW_P?
- 2) AI 回答 (Claude Code + DeepSeek-V4)：建议在 walk() 中给非叶 PTE 添加 PTE_HW_P（定义 $PTE_V_DIR = PTE_V \mid PTE_HW_P$ ）。同时添加 dbar 屏障在 sfence_vma() 中以确保 PTE 写入对 lddir 可见。
- 3) 开发者采纳实施：修改 walk() 中非叶 PTE 为 PTE_V_DIR，在 sfence_vma 和 hal_trampoline.S 各 invtlb 前添加 dbar 0。结果 init 进程 fork 立即崩溃 (P=1 改变了 lddir 行为)，dbar 也无法修复数据不一致。
- 4) 开发者的正确思路：查阅参考项目 xv6-loongarch-exp-main 确认参考项目也不在非叶 PTE 设 P 位。结论是手册“P 在 walk 中使用”的说法在不同 QEMU 实现中行为不一致，此方向不是根因，放弃该路线。

交互 2：vmfault 手工填 TLB（方向性错误）

- 1) 开发者提问：能否绕过 TLB refill handler，在 vmfault 页故障处理中直接使用 tlbwr 手工填 TLB?
- 2) AI 回答 (Claude Code + DeepSeek-V4)：建议实现 tlb_fill_from_pte() 函数，将正确的 PTE 写入 TLBELO0/TLBELO1，通过 tlbsrch 查找已有条目，再用 tlbwr 覆写。同时建议在 kvmnithart 中设置 PWCL/PWCH 为 4 级配置。
- 3) 开发者采纳实施：实现 vmfault 手工 TLB 填充代码，添加 PWCL/PWCH 配置。结果发现两个致命问题：(a) vmfault 填的 TLB 条目被 userret 中的 invtlb 立即冲掉；(b) PWCL/PWCH 写入后原 3 级 handler 行为被破坏。去掉 invtlb 后系统不崩溃但无用户态输出。
- 4) 开发者的正确思路：认识到 vmfault 事后填 TLB 的方案无法工作 (userret 必然 flush TLB)。正确方向应该是修改 TLB refill handler 本身让它填对条目，而不是事后修补。

交互 3: fork-chk 隔离测试 + copyinstr 追踪 (正确的调试方法)

- 1) 开发者提问: 无法确定问题是出在 TLB 映射还是页表建立。需要一种方法验证内核能否正确访问子进程物理页。
 - 2) AI 回答 (Claude Code + DeepSeek-V4): 建议三步隔离测试: (a) 在 fork 后通过 walkaddr 找到子进程 BSS 物理页, 用 DMW0 直接写入测试值并读回; (b) 在 copyinstr 中打印物理地址和字节值; (c) 在 vmfault 添加 debug printf 追踪是否触发 page fault。
 - 3) 开发者采纳实施: fork-chk 输出 wrote AB,CD read ab,cd —证明内核通过 DMW0 可正确读写子进程物理页; copyinstr 输出 pa0=0x638000 p[0]=0 p[1]=0 —证明内核读到的物理页数据为 0; vmfault 从未被调用—证明 BSS 页已映射, 没有触发 page fault。
- 结论: 内核页表正确, 但用户态通过 TLB 写入去了错误的物理地址。这明确指向 TLB refill handler 填入了错误 PPN 的根因方向。

交互 4: DMW 配置与参考项目对比

- 1) 开发者提问: 参考项目 xv6-loongarch-exp-main 使用 VSEG=0x9 + DMWIN_MASK, 我们使用 VSEG=0 恒等映射。是否需要对齐参考项目? 同时请确认我们的配置是否符合官方 QEMU 标准。
- 2) AI 回答 (Claude Code + DeepSeek-V4): 分析对比了两个项目的内存布局 and QEMU 配置。结论是参考项目适配的是特殊 QEMU 配置 (RAM 在 0x90000000), 而标准 QEMU virt 的 RAM 在 0x00000000-0x0FFFFFFF。我们 VSEG=0 恒等映射对标准 QEMU 是正确的。
- 3) 开发者采纳实施: 确认配置正确, 未修改 DMW 配置。

交互 5: HPTW 测试 + TLBRBADV 发现

- 1) 开发者提问: 设 TLBREENTRY=0 能否判断 la464 是否有 HPTW? TLB handler 应该读哪个 CSR 获取故障 VA?
 - 2) AI 回答 (Claude Code + DeepSeek-V4): (a) 设 TLBREENTRY=0 测试 HPTW —若系统不崩溃则 HPTW 在工作; (b) 对比手册发现 TLB refill handler 应读 TLBRBADV (CSR 0x89) 而非 BADV (CSR 0x7)。
 - 3) 开发者采纳实施: TLBREENTRY=0 测试结果: 系统不崩溃但 hang, 确认 HPTW 可用但填充的条目与预期不同; TLBRBADV 修复被最终方案采纳。
- 结论: 两个建议都对最终解决有帮助。

交互 6：TLB refill handler 3 级 ld.d 软 walk（最致命的错误方向）

- 1) 开发者提问：当前的 TLB handler 使用 ld.d 做软件 walk，从 PGDL 直接走 $L2(VA[38:30]) \rightarrow L1(VA[29:21]) \rightarrow L0(VA[20:12])$ ，共 3 级。copyinstr2 测试失败是否与 walk 级数有关？
- 2) AI 回答 (Claude Code + DeepSeek-V4)：分析的结论是“3 级 walk 与 4 级页表不匹配”，但提出的修复方案始终是在软件 walk 框架内修补——尝试了 ld.d 4 级 walk（用 CSR 保存寄存器）、CSR 保存寄存器版本、bstrins.d 替代版本等。所有方案均因 DA=1 约束或 bstrins.d 兼容性问题而失败。始终未建议使用 lddir/ldpte 指令替代 ld.d 软 walk。
- 3) 开发者采纳实施：AI 提出的多种软件 walk 变体均未成功。系统始终在 init 首次 TLB miss 时崩溃 (scause=0xc)。
- 4) 开发者的正确思路：开发者自行查阅 LoongArch 手册 §4.2.5（软件页表 walk 指令），发现手册规定的标准做法是使用 lddir/ldpte 指令而非 ld.d。改用 lddir/ldpte 重写 TLB handler + 在 kvmminihart 显式配置 PWCL/PWCH + ldpte 后清除 P/W 保留位，从根本上解决了问题。这也是整个项目最关键的突破——从“自己实现软 walk”转变为“使用架构标准指令”。

交互 7：MAXVA 问题定位与解决

- 1) 开发者提问：usertests -q 中 MAXVAplus 测试失败，用户态对高地址 ($\geq 1 \ll 38$) 的写入没有稳定失败。怀疑是 LoongArch 48-bit VA 与 xv6 39-bit 契约冲突。
- 2) AI 回答 (Claude Code + DeepSeek-V4)：分析了 LoongArch 硬件 VA 截断导致的地址别名问题，建议恢复 MAXVA=1 $\ll$ 38、在 TLB handler 入口加边界检查、exec 侧加低页 guard、通过 RVACFG 降低硬件有效 VA 的综合方案。
- 3) 开发者采纳实施：开发者在 Codex 协助下实现了完整的三层防护方案，MAXVAplus 测试通过，usertests -q 全量通过。

交互 8：根因分析总结

开发者请求：在项目完成后，要求 AI 对比 git diff 分析之前版本与最终版本的所有差异，找出之前长期无法解决的根本原因。

AI 分析结果：对比 commit 765c76f (broken) 与 c9e1c32 (fixed)，发现之前版本存在以下 6 个根本性错误：

1. 3 级 ldd 软 walk 与 4 级页表不匹配（最致命——整个 walk 偏移一级，读到错误 PPN）
2. 未清除 P/W 保留位直接写入 TLBELO（污染 TLB 条目）
3. 读 BADV (0x7) 而非 TLBRBADV (0x89)
4. vmfault 填 TLB 方案被 userret invtlb 冲掉（方向性错误）
5. MAXVA 从 $1 \ll 38$ 放宽到 $1 \ll 46$ 破坏 xv6 地址契约
6. PWCL/PWCH 未显式配置

其中第 1 点是所有问题的根因，AI 在开发过程中始终未提出改用 lddir/ldpte 的正确方案。

7 附录：HAL 文件清单

表 6: LoongArch HAL 实现文件 (hal/loongarch/)

文件	行数	功能
arch.h	462	CSR 定义、PTE 宏、RISC-V 兼容层、estat_to_scause、RVACFG
memlayout.h	66	QEMU virt 物理地址布局、TRAMPOLINE/-TRAPFRAME/KSTACK/USER_TOP
hal_entry.S	18	启动入口，设栈后跳 start()
hal_start.c	89	flash→RAM 重定位、多核同步、FPU 使能
hal_tlbrefill.S	78	TLB 重填处理器 (lddir/ldpte 4 级 walk + P/W 清除 + MAXVA 检查)
hal_tramp.S	140	uservec/userret 陷入/返回 (KSave0/1 替代 sscratch)
hal_kvec.S	63	kernelvec 内核态中断向量 (17 寄存器保存/恢复)
hal_swtxh.S	41	12 寄存器上下文切换
hal_intr.c	90	EIOINTC 中断控制器 (ISR 位扫描)
hal_uart.c	106	16550a 串口驱动 (QEMU 兼容模式)
hal_virtio.c	130	RAM-disk 块设备驱动 (支持双盘)
kernel.ld	55	链接脚本 (flash/RAM 分离, .tlbrefill/trampsec 定址)
user.ld	39	用户程序链接脚本 (从 0x2000 起始)
合计	1377	